

STEERING LANGUAGE MODELS WITH ACTIVATION ENGINEERING

Alexander Matt Turner
Independent researcher
alex@turntrout.com

Lisa Thiergart
MIRI

Gavin Leech
University of Bristol
g.leech@bristol.ac.uk

David Udell
Independent researcher

Juan J. Vazquez
Arb Research

Ulisse Mini
MATS

Monte MacDiarmid
Anthropic

ABSTRACT

Prompt engineering and finetuning aim to maximize language model performance on a given metric (like toxicity reduction). However, these methods do not fully elicit a model’s capabilities. To reduce this gap, we introduce *activation engineering*: the inference-time modification of activations in order to control (or *steer*) model outputs. Specifically, we introduce the *Activation Addition* (ActAdd) technique, which contrasts the intermediate activations on prompt pairs (such as “Love” versus “Hate”) to compute a *steering vector* (Subramani et al., 2022). By tactically adding in e.g. the “Love” – “Hate” steering vector during the forward pass, we achieve SOTA on negative-to-positive sentiment shift and detoxification using models including LLaMA-3 and OPT. ActAdd yields inference-time control over high-level output properties (like topic and sentiment) while preserving performance on off-target tasks. ActAdd is lightweight: it does not require any machine optimization and works with a single pair of data points, which enables rapid iteration over steering. ActAdd demonstrates the power of activation engineering.

1 INTRODUCTION

LLMs contain hidden capabilities we do not know how to fully elicit (Korinek, 2023). Naively prompting a model with a question does not maximize the probability of the correct response. Carefully consider how prompting a model to think “step-by-step” (Wei et al., 2022) massively improves performance on a range of benchmarks. Similarly, “few-shot” prompting a model with correct answers to unrelated in-distribution questions allows “in-context learning” for e.g. stronger performance on NLP tasks (Brown et al., 2020). Importantly, these interventions do not supply the LLM with extra task-relevant information or update the algorithm implemented by the LLM’s computational graph. Even though the model is initially *able* to score higher on these benchmarks, those capabilities do not emerge without a specific intervention. We therefore hypothesize the presence of an *elicitation overhang*: we do not know how to elicit all relevant abilities and information from frontier models.

Prompt engineering is the most obvious way to steer a model, but prompting has limited reliability (Ye & Durrett, 2022; Wang et al., 2024). Therefore, to reduce the elicitation overhang, we explore a new modality for steering language model outputs. By strategically perturbing activations during the forward pass, we hope to more reliably and effectively steer models compared to prompt engineering. We call this methodology *activation engineering*.

We suspect that compared to prompt engineering, activation engineering can elicit a wider range of model capabilities. Consider, for example, a model optimized to imitate the text outputs of eloquent poets and awkward mathematicians. The model may contain the internal mechanisms required to output text which is *both* eloquent and mathematical. However, if the model is an accurate estimator of the training distribution, it will (correctly) assign low probability to eloquent mathematical prose. Because nothing in the training data was both eloquent and mathematical, there may exist no prompt which elicits mathematical prose. In contrast, activation engineering might be able to simultaneously activate the circuitry for eloquent speech and for mathematical content.

To demonstrate the power of activation engineering, we introduce *Activation Addition* (ActAdd). Suppose we want to achieve negative-to-positive sentiment control (Li et al., 2018; Dathathri et al., 2020). To achieve this, ActAdd first compares the model’s activations on a contrast pair of prompts, such as the prompts “Love” and “Hate.” The otherwise-similar prompts differ along the target dimension of sentiment. ActAdd then computes the difference of these activations in order to compute *steering vectors*. These vectors act like “virtual bias terms” because ActAdd *directly adds* the steering vectors to the forward pass at inference time. By shifting the inference-time activations along the direction of the steering vector, ActAdd steers the model to generate positive sentiment completions (Table 1).

Table 1: The impact of ActAdd. The steering vectors are computed from (“Love” - “Hate”) and (“I talk about weddings constantly” - “I do not talk about weddings constantly”). Appendix Table 6 shows more examples.

Prompt	+	steering	=	completion
I hate you because...		[None]		...you are the most disgusting thing I have ever seen.
		ActAdd (love)		...you are so beautiful and I want to be with you forever.
I went up to my friend and said...		[None]		...“I’m sorry, I can’t help you.” “No,” he said. “You’re not.”
		ActAdd (weddings)		...“I’m going to talk about the wedding in this episode of Wedding Season. I think it’s a really good episode. It’s about how you’re supposed to talk about weddings.”

Contributions. We unify past literature on related topics to introduce *activation engineering*. To better elicit the full capabilities of models, we introduce the ActAdd steering method, which achieves SOTA on toxicity reduction and sentiment control. We thoroughly test the steered models to verify the preservation of their general capabilities. We therefore show the promise of ActAdd as an effective and cheap method for steering LLM outputs.

2 RELATED WORK

Latent space arithmetic. Research in generative models for computer vision has long demonstrated the ability to steer image generation using derived vectors, including steering latent variables – most famously, shifting activations along a direction that corresponds to smiles in images (Larsen et al. 2016; White 2016). Similarly, in the text domain, classic results on the word2vec embedding show that arithmetic on word vectors can capture some parts of semantic reasoning (for instance, analogies: Mikolov et al. 2013b;a). Our work focuses on steering generative language models.

LLM steering. Many approaches attempt to affect the output of a pretrained LLM, whether:

- *Intervening on weights*, as with supervised finetuning, RLHF, steerable layers, and weight editing (that is, targeted fine-tuning) (Ranzato et al. 2016; Ziegler et al. 2019; Dathathri et al. 2020; Meng et al. 2023; Ilharco et al. 2023). However, naive RLHF, finetuning, and weight editing have known side-effects on overall model performance (Hase et al. 2023; Qi et al. 2023; Brown et al. 2023);

- *Intervening at decoding*, as with guided or trainable decoding (Gu et al. 2017; Grover et al. 2019; see Zhang et al. 2022a for an overview of controlled generation and Jin et al. 2022 for textual style transfer);
- *Intervening on the prompt*, as with automated prompt engineering (Shin et al. 2020; Zhou et al. 2022);
- *Intervening on token embeddings*, as with ‘soft prompting’ (Li & Liang 2021; Lester et al. 2021; Khashabi et al. 2022);
- *Intervening on activations*, for instance by freezing the weights of the LLM and searching for a ‘steering vector’ of activations, e.g. using gradient descent (Subramani et al. 2022; Hernandez et al. 2023). These optimized extraction methods, which search for a steering vector, differ from extraction methods which directly compute it (present work and Li et al. 2023b). In our work, we do not use gradient descent or other optimization methods.

Table 2: Locating our work in the steering literature.

Intervention vectors obtained via	Vector intervenes on model ...	
	... weights	... activations
Differences after fine-tuning	Ilharco 2023	N/A
Per-query gradient-based search	Meng 2022, Orgad 2023	Dathathri 2020 Subramani 2022 Hernandez 2023
Differences between prompt pairs	N/A	ActAdd (present work), Li et al., 2023b

Activation engineering. Activation engineering involves creating vectors of activations which cause desired changes to output text when added to the forward passes of a frozen LLM (Dathathri et al. 2020). Table 2 organizes prior work by intervention type.

An early antecedent is the Plug-and-Play Language Model of Dathathri et al. 2020. This uses a separate classifier (one classifier per attribute to steer towards) to perturb the model’s activations to generate text that accords more closely with the classifier’s target. Subramani et al. 2022 extract latent steering vectors from a frozen LLM, successfully discovering sentence-specific vectors which steer completions to near-perfect BLEU scores (i.e. control of the LLM’s generation) and unsupervised style transfer. However, the method requires running gradient descent for each new steering vector. Hernandez et al. 2023 locate and edit an LLM’s knowledge through learning an encoding of facts in its activation space. Ablating attention heads can also be seen as activation engineering, though the technique is mostly used for model interpretation rather than steering (Michel et al. 2019; Olsson et al. 2022).

Independently, Li et al. 2023b developed a similar method called ITI which computes steering vectors which are selectively applied according to trained linear probes. They use these probes to find attention heads with different activation distributions for true and false statements. They steer the model toward truthful outputs, where our experiments cover a range of goals. In addition, ITI adds the same steering vector at all sequence positions during inference and ITI requires dozens of samples. In contrast, ActAdd we add steering vectors to a subset of sequence positions and require as few as 2 samples. Similar work on ‘in-context vectors’ also followed ours (Liu et al. 2023). Lastly, Zou et al. 2023’s “representation engineering” also followed our work. They develop a range of techniques for deriving steering vectors and for steering models using activation-space edits and optimization. In comparison to Zou et al. 2023, we steer different models (LLaMA-3, OPT, GPT-2, and GPT-J) on different tasks (detoxification and sentiment control).

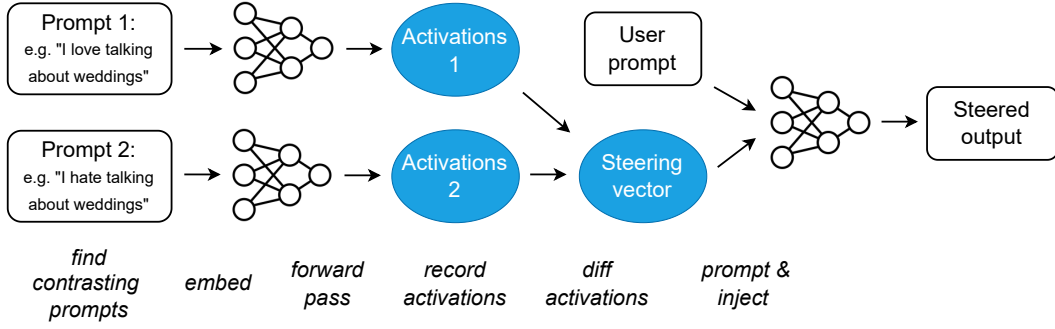


Figure 1: Schematic of the Activation Addition (**ActAdd**) method. $\circ$ = natural language text; $\bullet$ = vectors of activations just before a specified layer. In this example, the output is heavily biased towards discussing weddings, regardless of the topic of the user prompt. (See Algorithm 1 for the method’s parameters: intervention strength, intervention layer, and sequence alignment.)

3 HOW ACTIVATION ADDITION WORKS

We use decoder-only Transformer neural networks (Vaswani et al. 2017). The LLMs in this work contain a stack of Transformer layers, each consisting of multi-head attention (MHA) and a feedforward network (FFN). We focus on its “residual streams” (Elhage et al. 2021), the sequences $(\mathbf{x}_0, \dots, \mathbf{x}_n)$ of intermediate activation vectors processed by each layer. ActAdd manipulates the residual stream values $\mathbf{h}^l$ input to layer l . Each layer performs MHA and FFN computations on $\mathbf{x}_i$, adding $\mathbf{x}_{i+1}$ to the stream. The final vector $\mathbf{x}_n$ in the stream can then be decoded into the next-token prediction. At inference time, the residual stream is initialized $\mathbf{h}^1$ with the embedding of the tokenized prompt.

Activation addition. Our method takes a pair of natural-language prompts (p_+, p_-) , where p_+ represents the property we wish output text to emphasise (e.g. love) and p_- represents its opposite (e.g. hate or indifference). $\mathbf{h}_+^l$ is the activation vector for the prompt p_+ at layer l . The difference $\mathbf{h}_+^l - \mathbf{h}_-^l$ is a new activation vector which (intuitively) captures the difference between a prompt with the target property, and a prompt without it. The steering vector is computed before inference time.

Algorithm 1 ActAdd, optimization-free activation addition

Input: (p_+, p_-) = steering prompt pair, tokenized
 p^* = user prompt
 l = target layer
 c = injection coefficient
 a = sequence position to align $\mathbf{h}_A$ and $\mathbf{h}_{p^*}$
 M = pretrained language model

Output: S = steered output

```

 $(p'_+, p'_-) \leftarrow \text{pad\_right\_same\_token\_len}(p_+, p_-)$ 
 $\mathbf{h}_+^l \leftarrow M.\text{forward}(p'_+).\text{activations}[l]$ 
 $\mathbf{h}_-^l \leftarrow M.\text{forward}(p'_-).\text{activations}[l]$ 
 $\mathbf{h}_A^l \leftarrow \mathbf{h}_+^l - \mathbf{h}_-^l$ 
 $\mathbf{h}^l \leftarrow M.\text{forward}(p^*).\text{activations}[l]$ 
 $S \leftarrow M.\text{continue\_forward}(c\mathbf{h}_A^l + \mathbf{h}^l @ a)$ 

```

To obtain a steering vector, we perform a forward pass on each prompt, record the activations at the given location in each pass, take the difference $\mathbf{h}_+^l - \mathbf{h}_-^l$, and then finally rescale this difference in activations by an ‘injection coefficient’ c . To steer, we add the resulting activation vector to the input of layer l and allow the forward pass to continue, and so obtain our steered output.¹ c represents the

¹See Appendix C for implementation details.